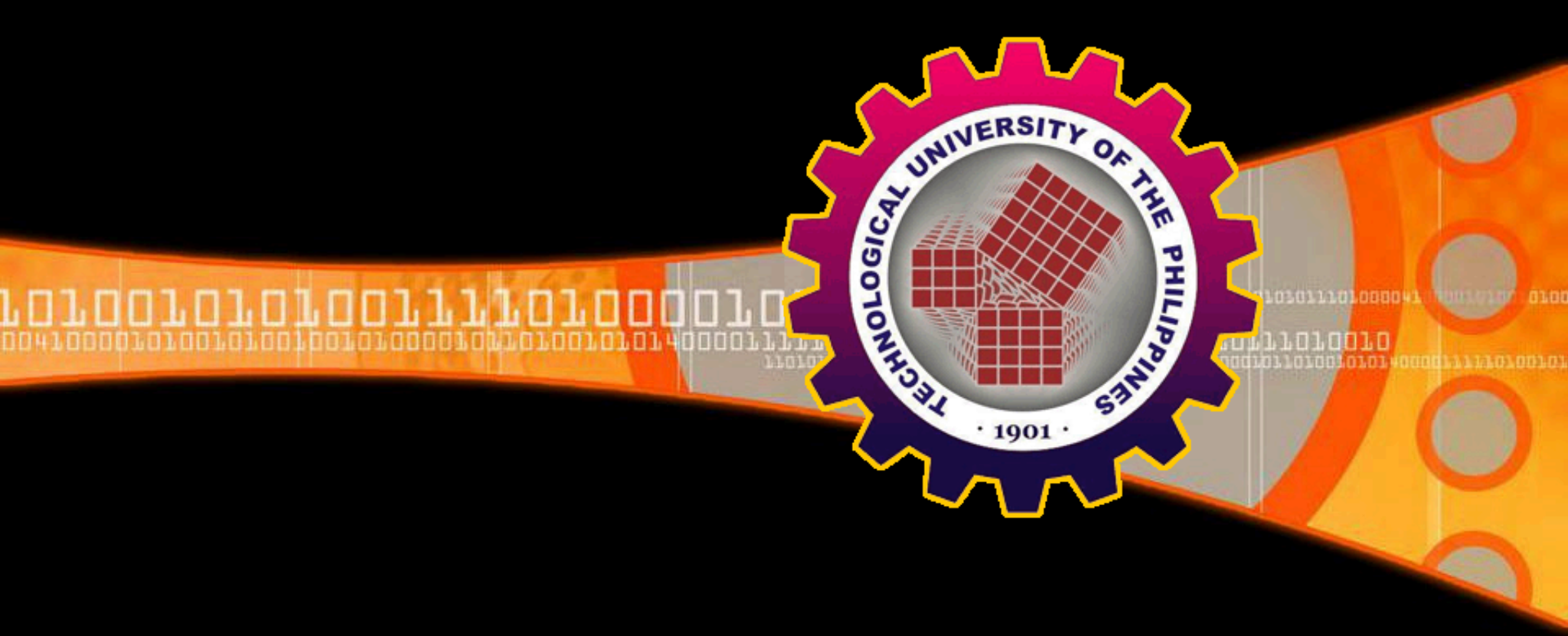


Remote Controlled Robot Planter

Bachelor of Science in Information Technology
Technological University of the Philippines - Taguig Campus



Canta, Aeron Jhon
Syparrado, Paul Dominic
Yumang, Jannella Marie Kaye

BSIT-S-2A

INTRODUCTION

Remote Controlled Robot Planter

The Remote Controlled Robot Planter is an Arduino-based farming assistant that automates seed planting, soil analysis, and watering. Equipped with sensors, actuators, and a microcontroller, it ensures optimal planting conditions while reducing manual labor. With Wi-Fi/Bluetooth control and a four-wheel chassis, it efficiently operates in farms and gardens, making agriculture smarter and more efficient.

OBJECTIVE

- Develop an Arduino-based robotic system for automated seed planting.
- Integrate soil moisture and pH sensors to ensure optimal planting conditions.
- Implement automated watering and seed dispensing mechanisms.
- Enable remote control via Wi-Fi/Bluetooth for easy operation.
- Enhance efficiency in agriculture and home gardening by reducing manual labor.

MATERIALS



















CONCLUSION



ph soil sensor





water pump relay



soil moisture sensor



2.4 GHz ESP-32





dc motor auger driller

4 wheel chassis



SG90 Mirco Servo

INPUT SENSORS

Soil Moisture Sensor – Measures the moisture level in the soil to determine if watering is needed. If the soil is dry, the system activates the water pump; if it is wet, the user is notified to proceed with planting.

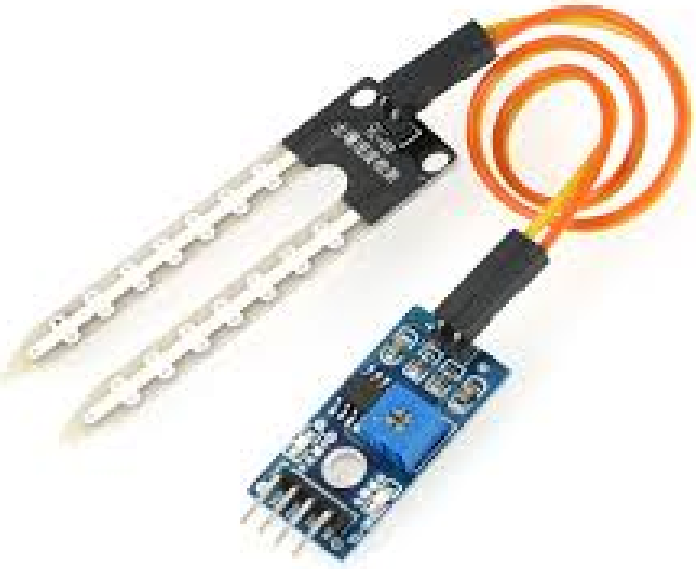
Soil pH Sensor – Analyzes the soil's pH level to ensure it is suitable for planting. If the pH is within the ideal range, the system dispenses seeds; if it is unsuitable, it alerts the user before proceeding.

Remote Control Input (Wi-Fi/Bluetooth) – Allows users to send movement and planting commands to the robot. It also provides real-time feedback on soil conditions and planting progress.

INPUT SENSORS

ph soil sensor





soil moisture sensor

android phone with bluetooth and wifi



OUTPUT COMPONENTS

Robot Movement – Controls the four-wheel chassis to navigate the planting area based on user commands. It moves forward, backward, and turns as needed.

Auger Drill (Digging Mechanism) – Uses a DC motor-powered auger drill to dig precise holes for planting seeds.

Seed Dispenser – Operated by a servo motor, it releases a seed into the hole after soil analysis confirms optimal conditions.

Water Pump – Activates to water the planted seed if the soil moisture level is too low. It is controlled by a relay module for safe operation.

OUTPUT SENSORS



servo motor seeder



4 wheel chas





waterpump relay





dc motor auger drill

THEORIES AND PRINCIPLES

Automation and Control Systems – The project applies embedded systems theory, where the ESP32/Arduino microcontroller processes sensor data and controls actuators to automate planting, watering, and movement.

Soil Science and Agronomy – The use of soil moisture and pH sensors is based on agricultural principles, ensuring seeds are planted in optimal soil conditions for better growth.

W**ireless Communication Principles** – The integration of Bluetooth and Wi-Fi enables remote operation, allowing users to control and monitor the robot in real time.

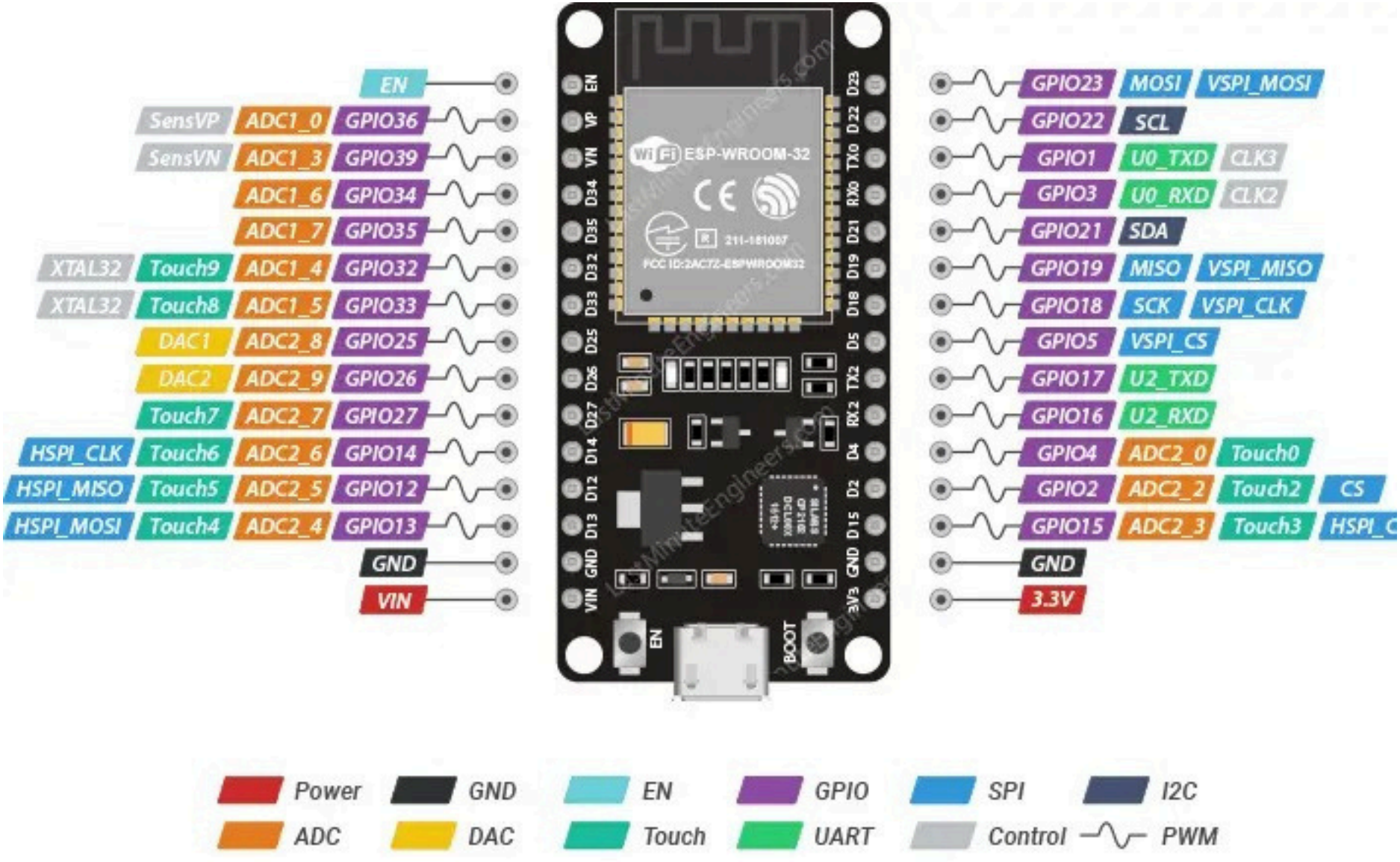
THEORIES AND PRINCIPLES

Mechatronics and Actuator Control – The project integrates mechanical, electronic, and computer systems to control components like DC motors, servo motors, and relay modules for precise movement, digging, seed dispensing, and watering.

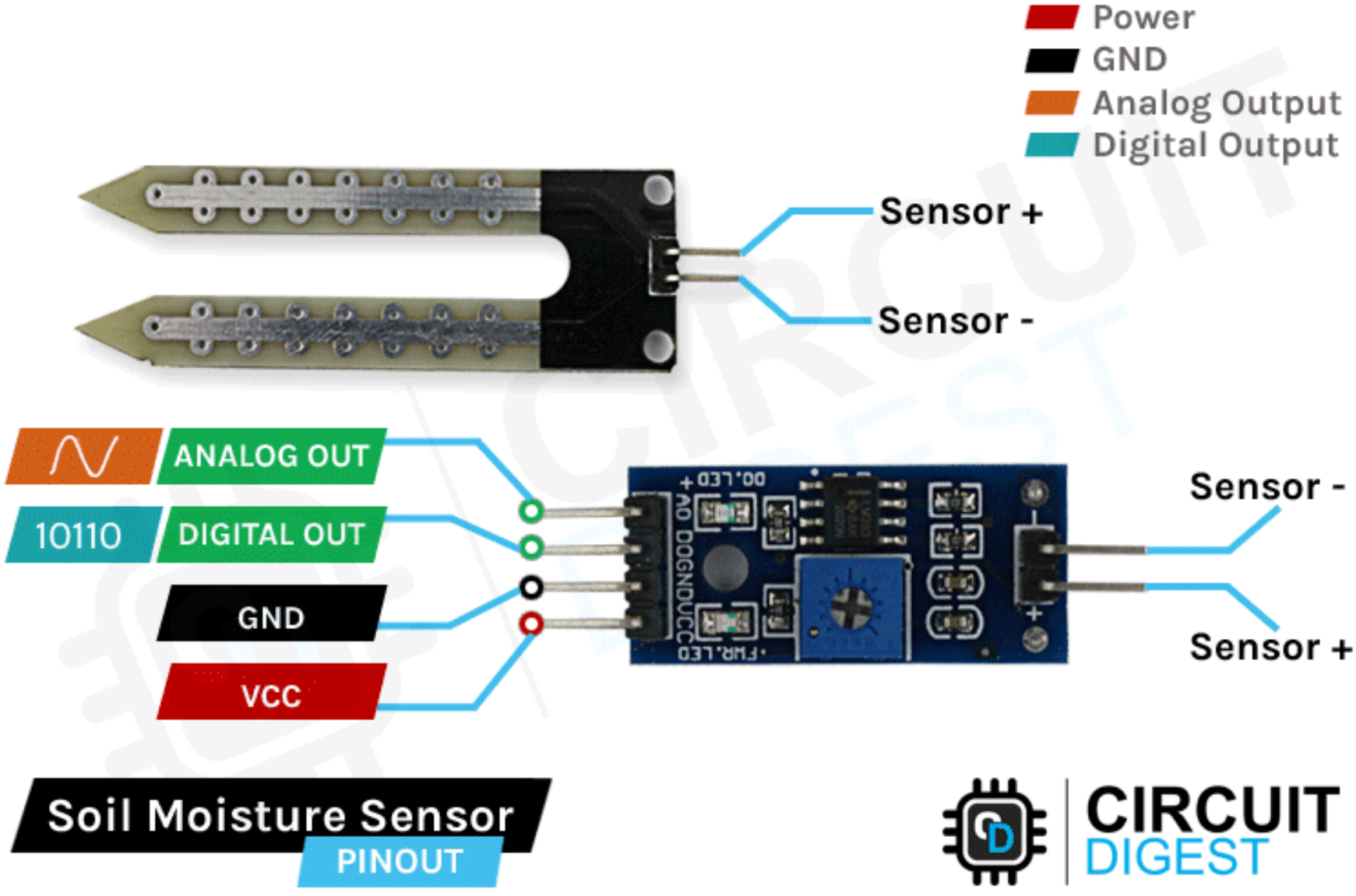
K**inematics and Motion Control** – The robot's movement is based on kinematics principles, ensuring proper navigation, turning, and speed regulation over different terrains while maintaining stability.

H**ydraulics and Fluid Mechanics** – The water pump system follows fluid mechanics principles to deliver the right amount of water to the soil, ensuring efficient irrigation and preventing overwatering.

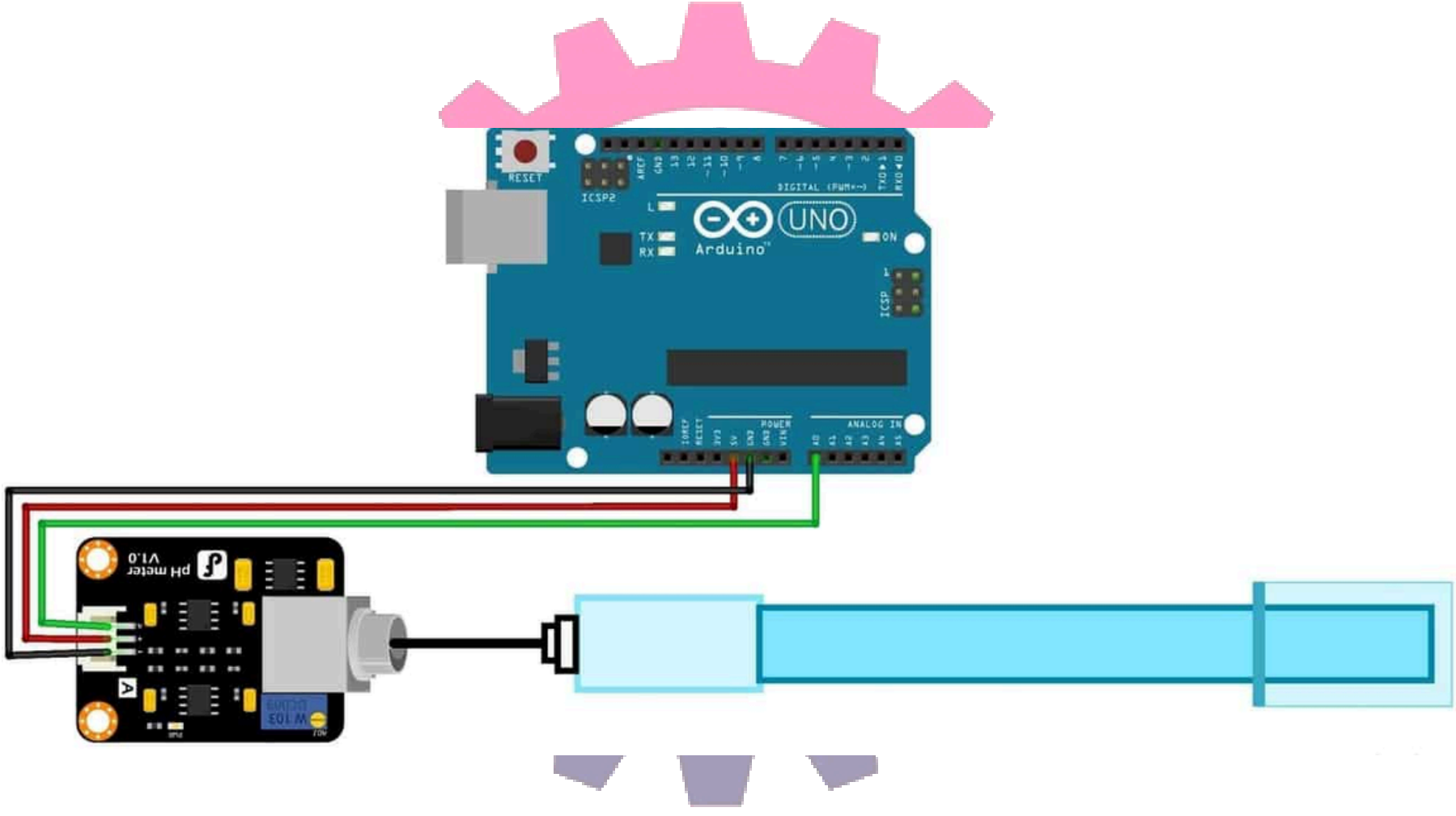
PIN CONFIGURATION



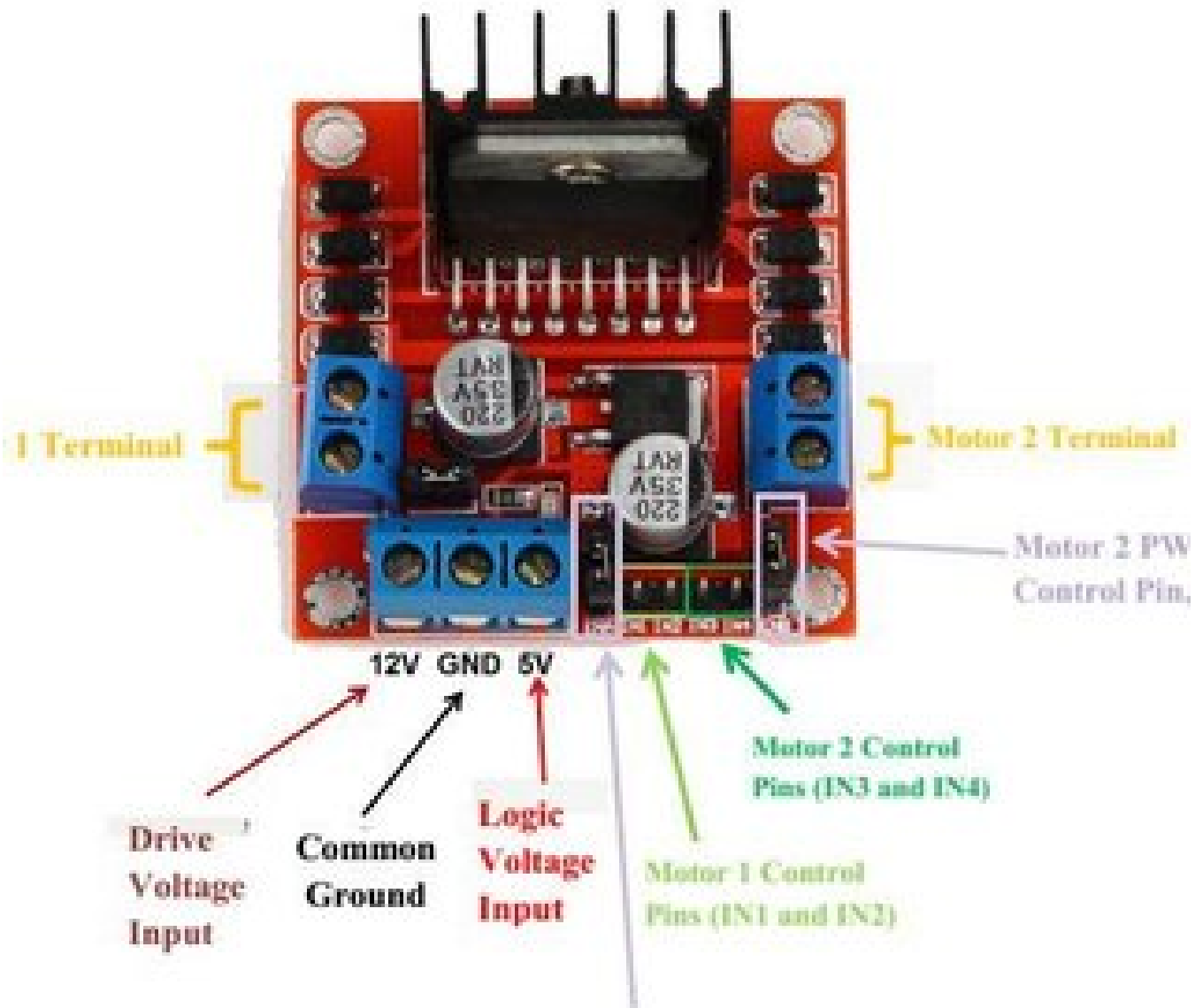
PIN CONFIGURATION



PIN CONFIGURATION



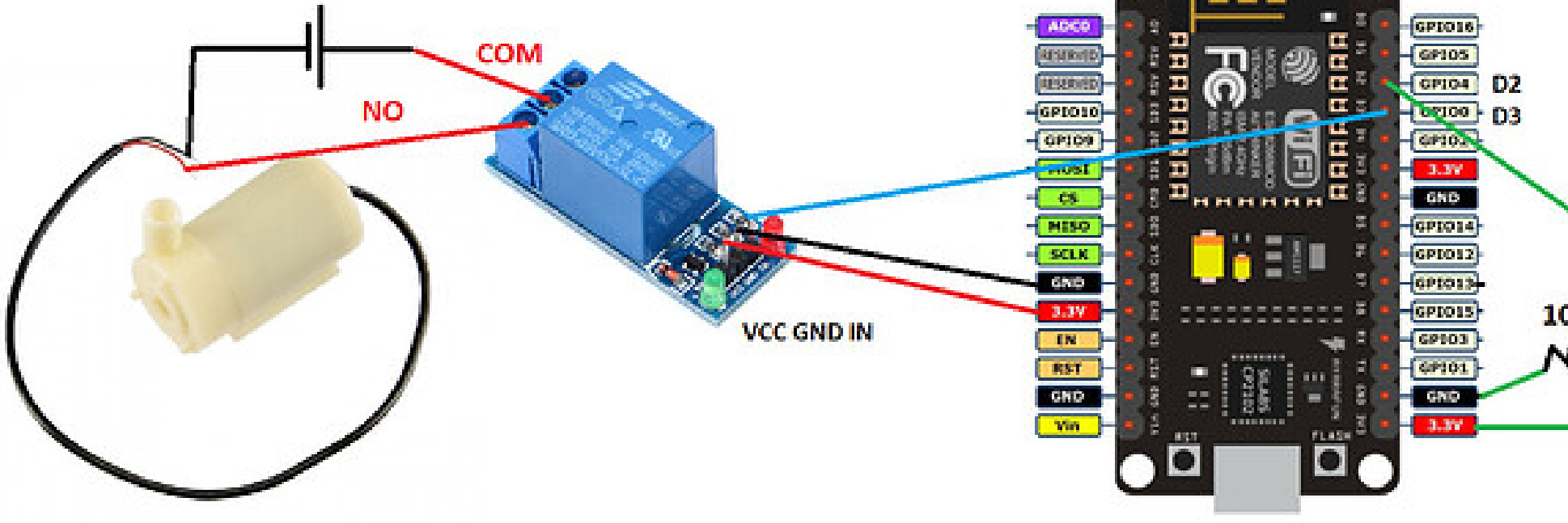
PIN CONFIGURATION



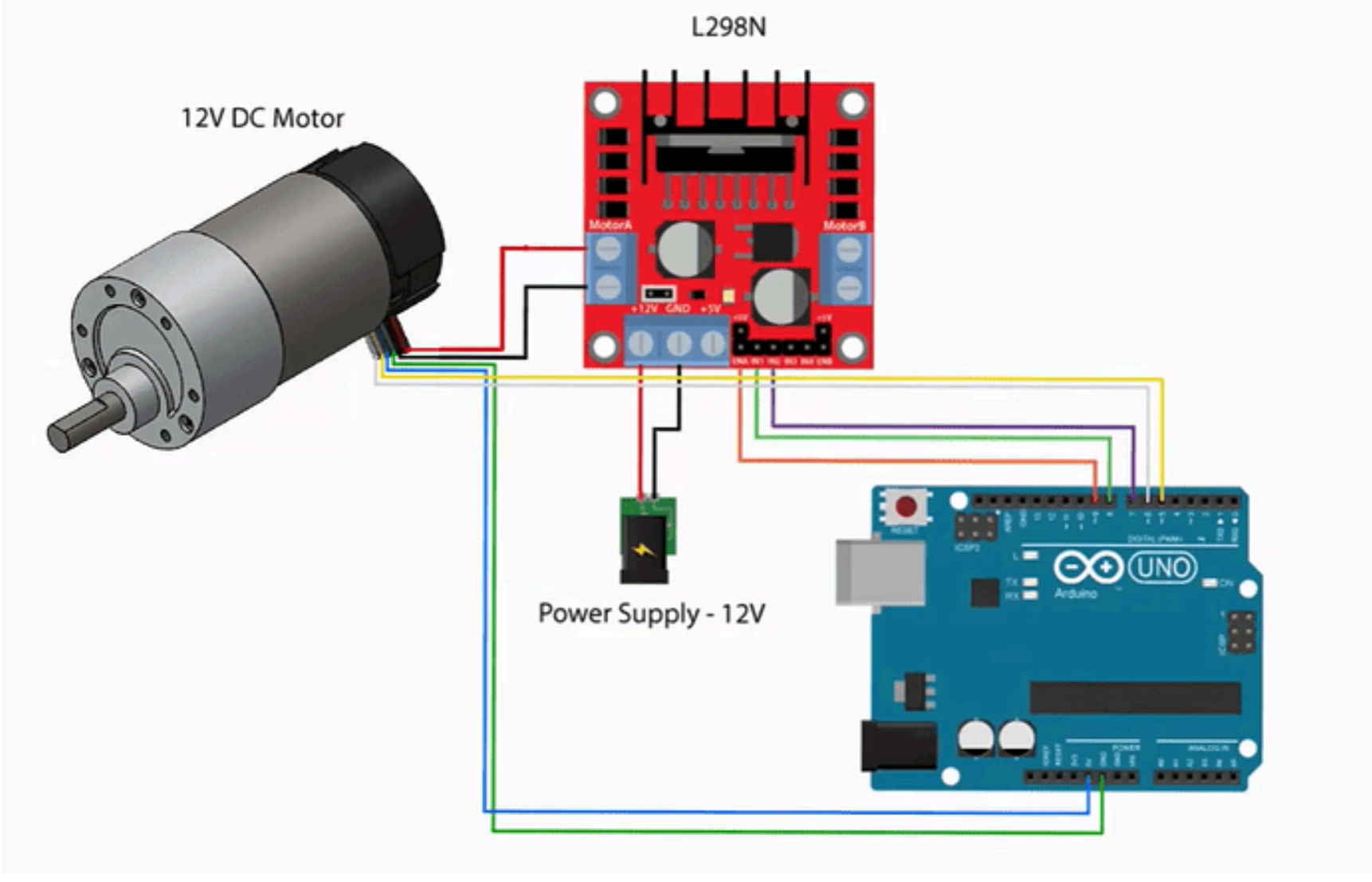
PIN CONFIGURATION



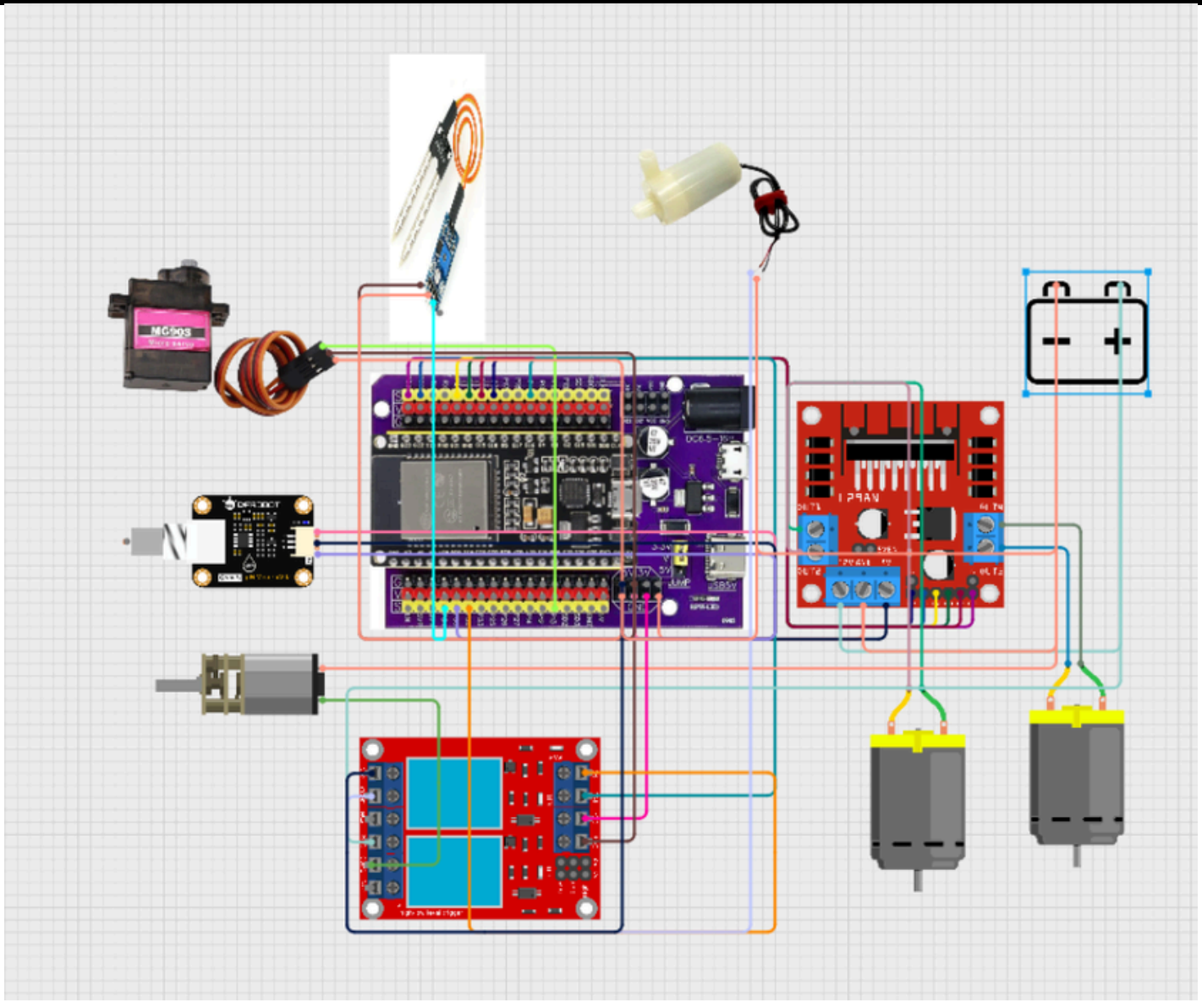
PIN CONFIGURATION



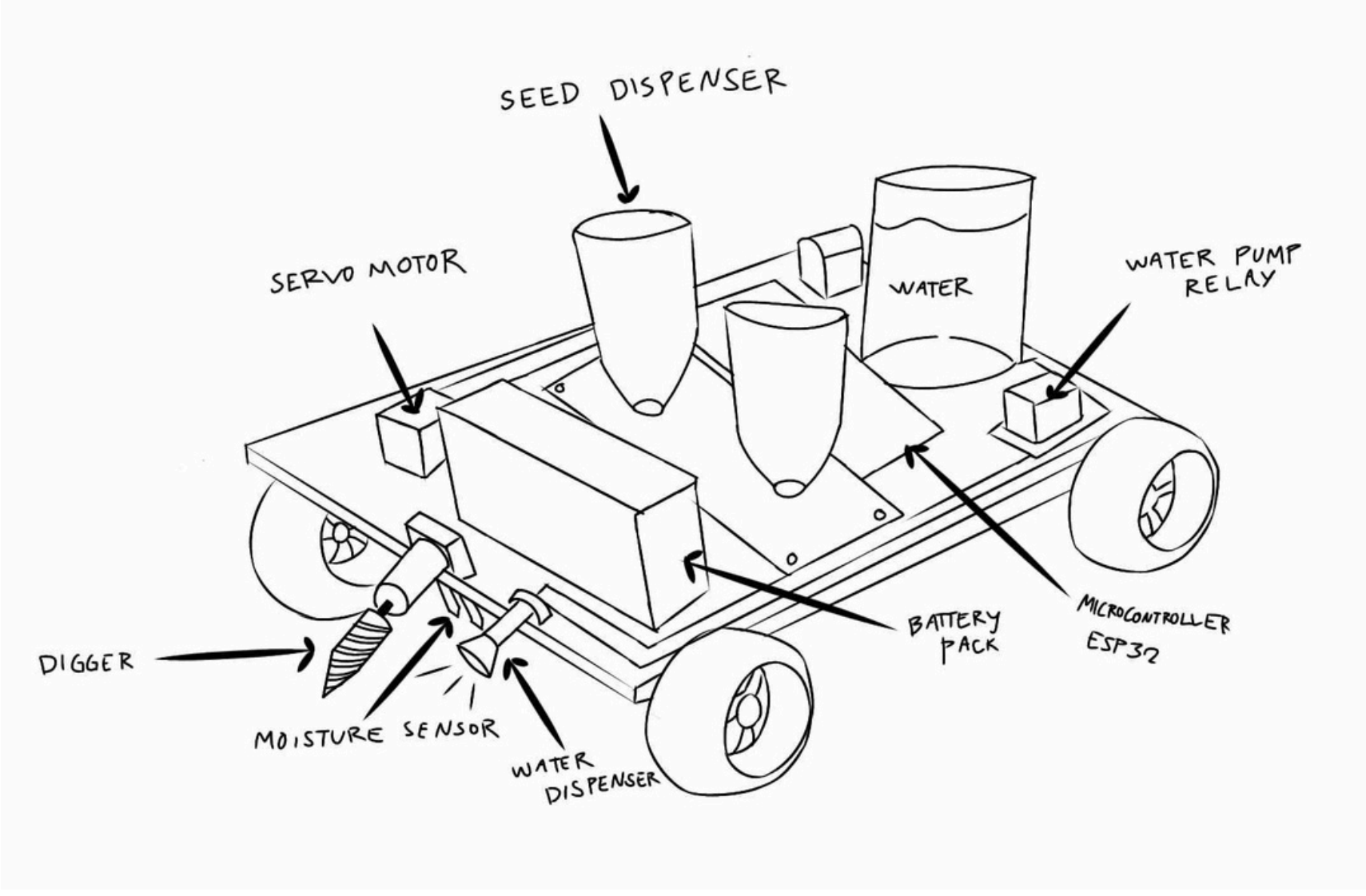
PIN CONFIGURATION



BLOCK DIAGRAM

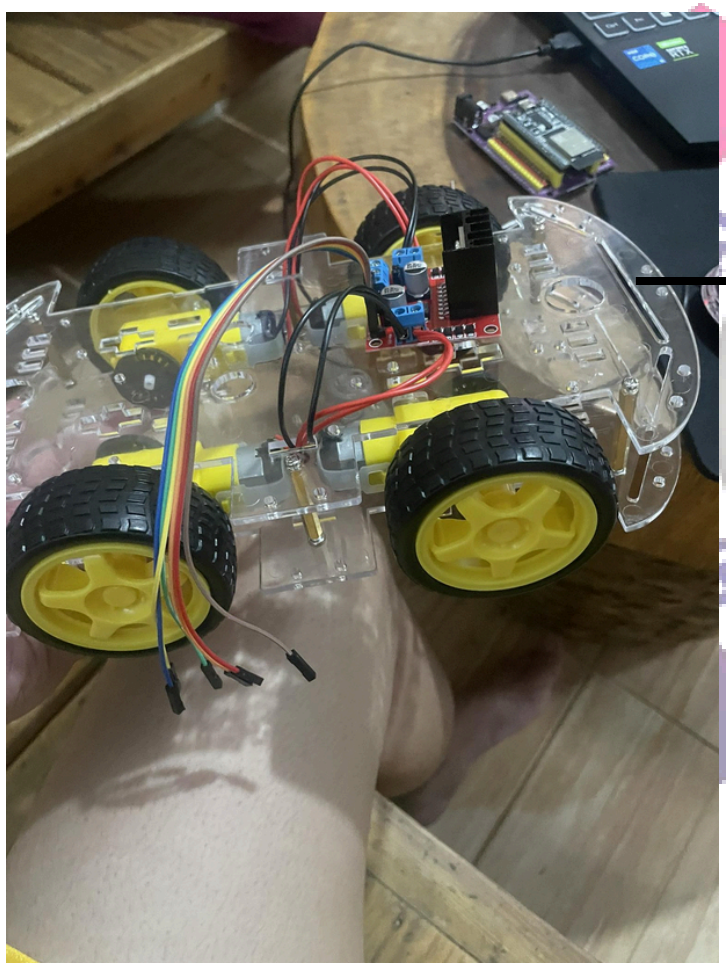


PROTOTYPE


SEED DISPENSER
SERVO MOTOR
WATER
WATER PUMP RELAY
DIGGER
MOISTURE SENSOR
WATER DISPENSER
BATTERY PACK
MICROCONTROLLER ESP32

MATERIAL TESTING

ESP32 and L298n (4 wheel chassis)

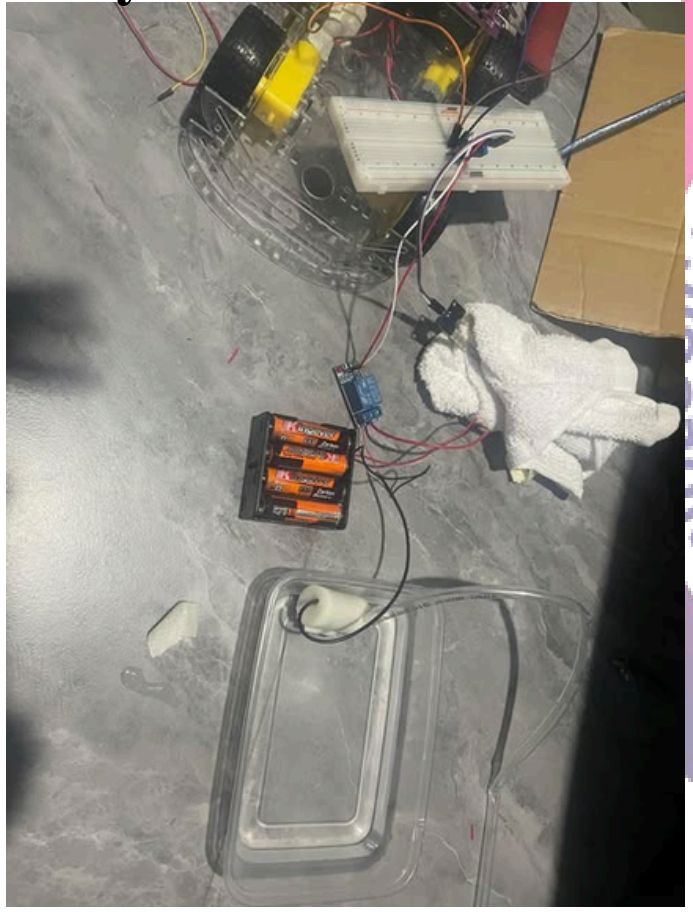


sketch_mar30a.ino

```
1  //esp32 board version must be 2.0.11
2  //otherwise the code will not compile
3
4  #include "BluetoothSerial.h"
5  #include <Arduino.h>
6  BluetoothSerial serialBT;
7
8  //Bluetooth signal Store in this variable
9  char btSignal;
10
11 //initial Speed
12 int Speed = 100;
13
14 //declare channel for pwm Output
15 #define R 0
16 #define L 1
17
18 //PWM Pin for Controlling the speed
19 int enA = 5;
20 int enB = 23;
21
22 //motor controlling pin
23 int IN1 = 22;
24 int IN2 = 21;
25 int IN3 = 19;
26 int IN4 = 18;
27
28 void setup() {
29   Serial.begin(115200);
30
31   //Bluetooth Name
32   serialBT.begin("esp32_car");
33
```

MATERIAL TESTING

Soil moisture sensor and waterpump relay



```
#define SOIL_SENSOR_PIN 34   // Soil moisture sensor (Analog)
#define WATER_PUMP_RELAY 4   // Relay to control the water pump

int moistureThreshold = 600;  // Adjust this value (0-4095 for ESP32 ADC)

void setup() {
  Serial.begin(115200);

  pinMode(WATER_PUMP_RELAY, OUTPUT);
  digitalWrite(WATER_PUMP_RELAY, LOW);  // Ensure pump is OFF initially
}

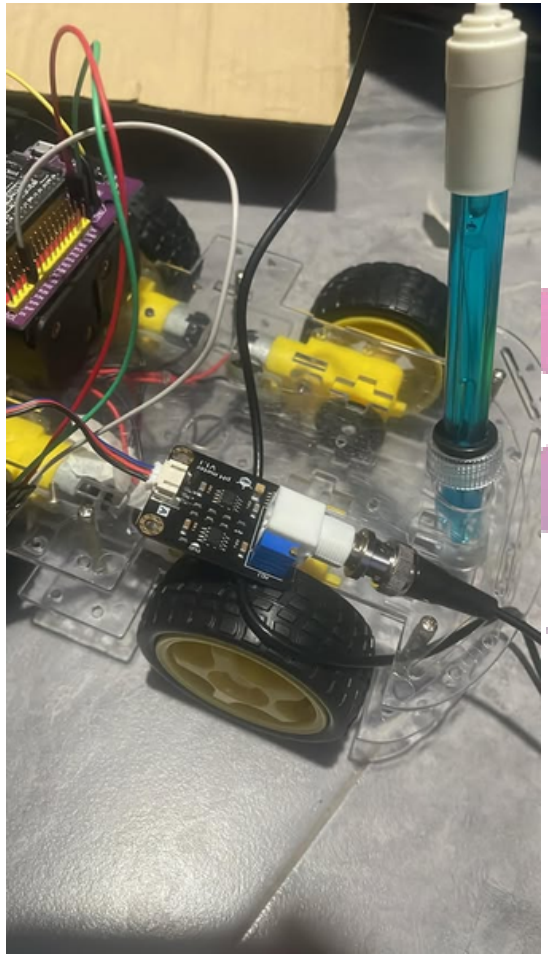
void loop() {
  int moistureValue = analogRead(SOIL_SENSOR_PIN);  // Read soil moisture
  Serial.print("Soil Moisture: ");
  Serial.println(moistureValue);

  if (moistureValue > moistureThreshold) {
    // Soil is DRY → Turn ON the water pump
    Serial.println("Soil is dry! Turning ON water pump.");
    digitalWrite(WATER_PUMP_RELAY, HIGH);
  } else {
    // Soil is WET → Turn OFF the water pump
    Serial.println("Soil is moist. Turning OFF water pump.");
    digitalWrite(WATER_PUMP_RELAY, LOW);
  }

  delay(2000);  // Wait for 2 seconds before the next reading
}
```

MATERIAL TESTING

PH sensor



sketch_mar30b.ino

```
#define PH_SENSOR_PIN 35  // pH Sensor (Analog Input)

// Calibration values (adjust based on your sensor)
#define PH_NEUTRAL 1500   // Approximate ADC value for pH 7.0
#define PH_ACIDIC 1200    // Lower than this is acidic
#define PH_ALKALINE 1800  // Higher than this is alkaline

void setup() {
  Serial.begin(115200);
}

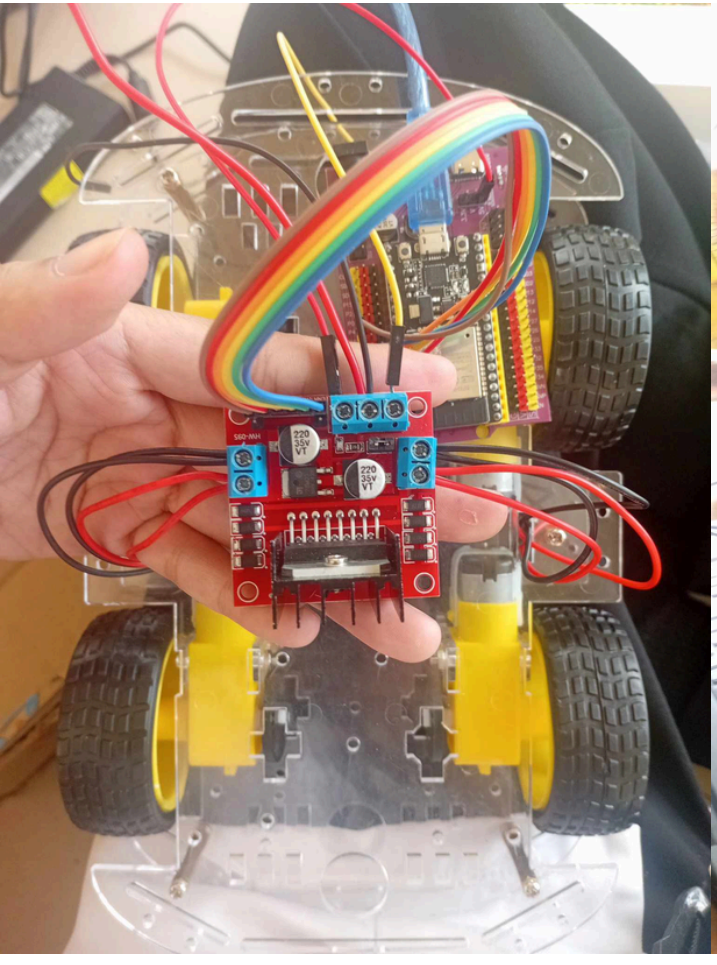
void loop() {
  int sensorValue = analogRead(PH_SENSOR_PIN);  // Read pH sensor
  Serial.print("pH Sensor Value: ");
  Serial.println(sensorValue);

  if (sensorValue < PH_ACIDIC) {
    Serial.println("⚠️ The soil is ACIDIC! (pH < 6.0)");
  }
  else if (sensorValue > PH_ALKALINE) {
    Serial.println("⚠️ The soil is ALKALINE! (pH > 8.0)");
  }
  else {
    Serial.println("✅ The soil pH is NEUTRAL (6.5 - 7.5).");
  }

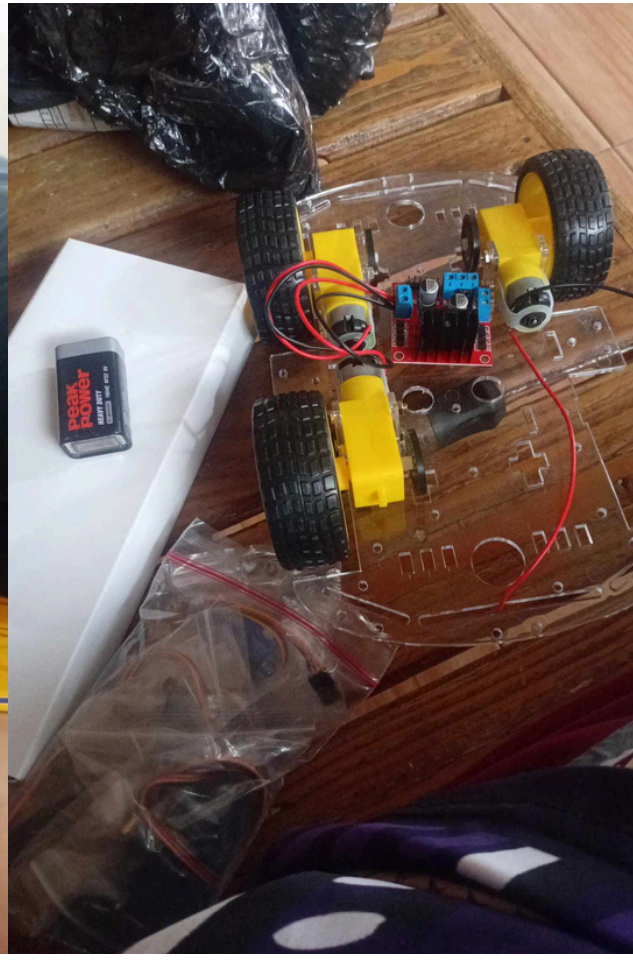
  delay(5000);  // Wait for 2 seconds before the next reading
}

```

ASSEMBLING

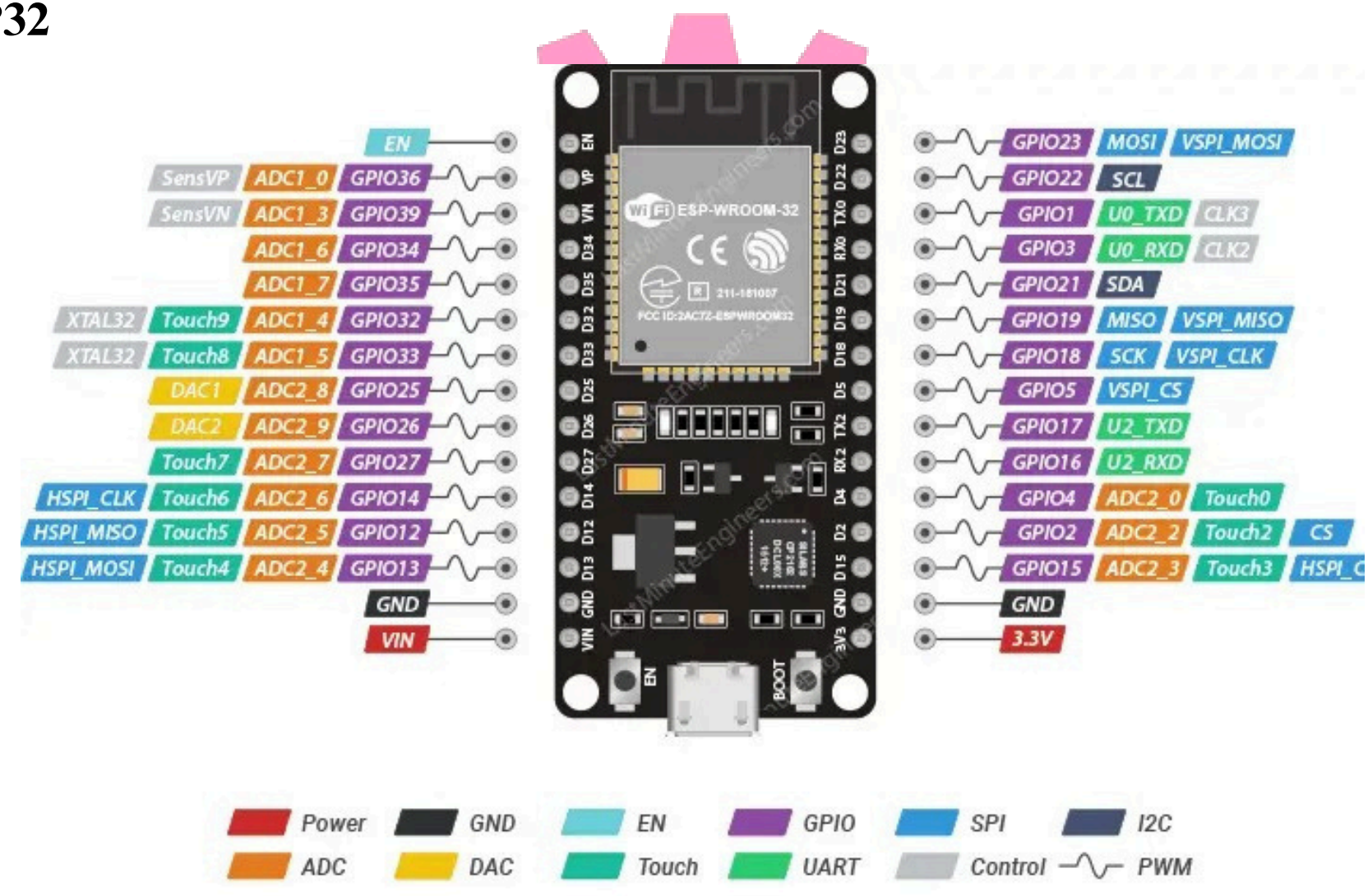






PINOUT DIAGRAM

ESP32



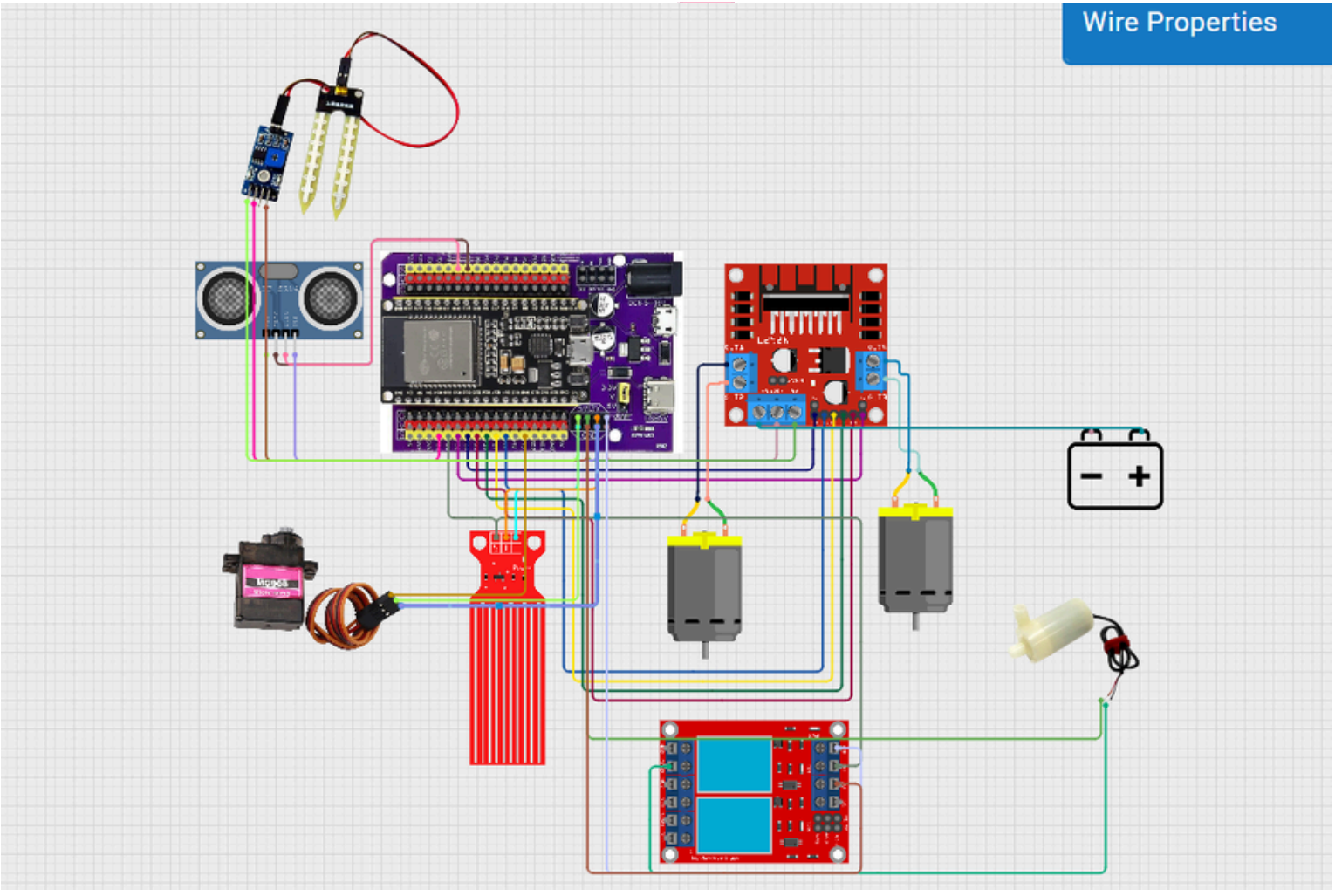
PINOUT DIAGRAM

Pin Name	Description	Pin Name	Description
3v3	3.3 V power supply	GND	Ground
EN	CHIP_PU, Reset	IO23	GPIO23
VP	GPIO36, ADC1_CH0, S_VP	IO22	GPIO22
VN	GPIO39, ADC1_CH3, S_VN	TX	GPIO1, U0TXD
IO34	GPIO34, ADC1_CH6, VDET_1	RX	GPIO3, U0RXD
IO35	GPIO35, ADC1_CH7, VDET_2	IO21	GPIO21
IO32	GPIO32, ADC1_CH4, TOUCH_CH9, XTAL_32K_P	GND	Ground
IO33	GPIO33, ADC1_CH5, TOUCH_CH8, XTAL_32K_N	IO19	GPIO19
IO25	GPIO25, ADC1_CH8, DAC_1	IO18	GPIO18

PINOUT DIAGRAM

IO26	GPIO26, ADC2_CH9, DAC_2	IO5	GPIO5
IO27	GPIO27, ADC2_CH7, TOUCH_CH7	IO17	GPIO17 3
IO14	GPIO14, ADC2_CH6, TOUCH_CH6, MTMS	IO16	GPIO16 3
IO12	GPIO12, ADC2_CH5, TOUCH_CH5, MTDI	IO4	GPIO4, ADC2_CH0, TOUCH_CH0
GND	Ground	IO0	GPIO0, ADC2_CH1, TOUCH_CH1, Boot
IO13	GPIO13, ADC2_CH4, TOUCH_CH4, MTCK	IO2	GPIO2, ADC2_CH2, TOUCH_CH2
D2	GPIO9, D2 2	IO15	GPIO2, ADC2_CH2, TOUCH_CH2
D3	GPIO10, D3 2	D1	GPIO15, ADC2_CH3, TOUCH_CH3, MTDO
CMD	GPIO11, CMD 2	D0	GPIO7, D0 2
5V	5 V power supply	CLK	GPIO6, CLK 2

PINOUT DIAGRAM

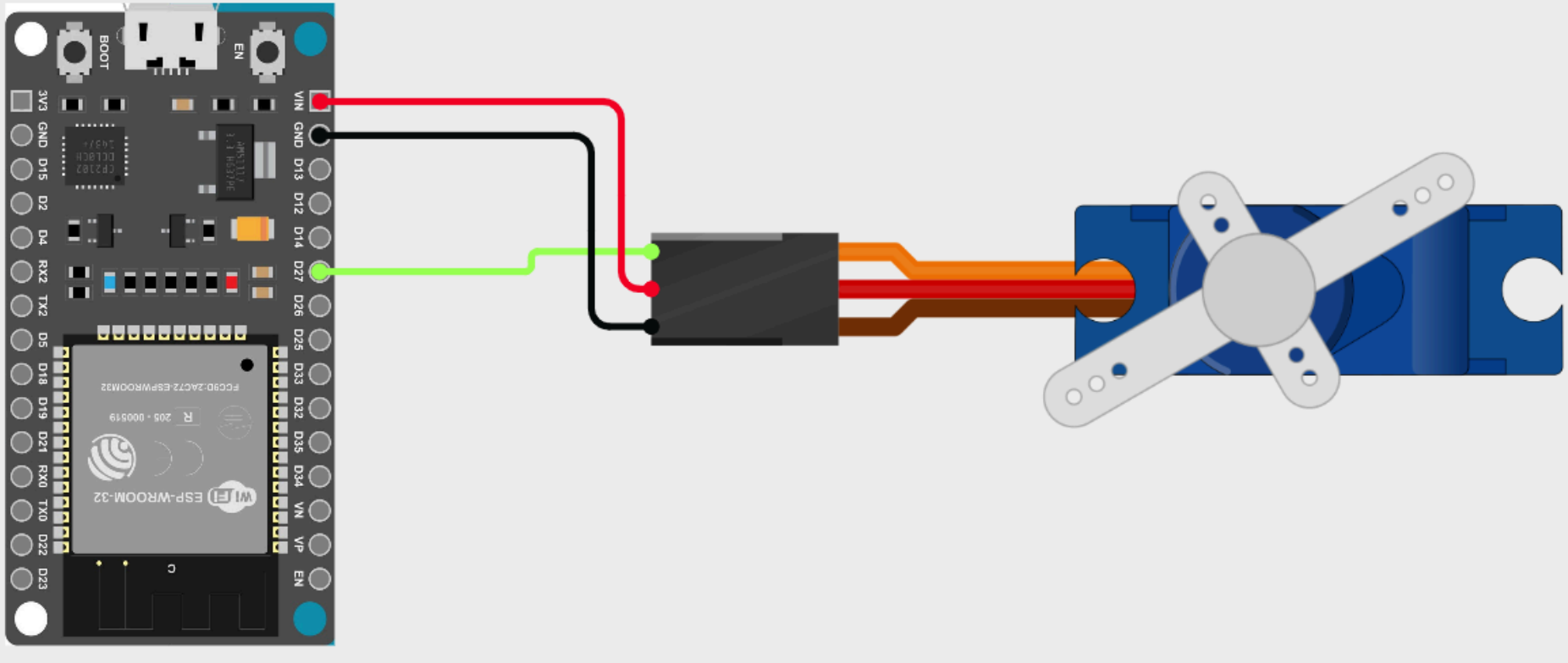


PINOUT DIAGRAM

Component	Pin on ESP32	Description
L298N Motor Driver		(For 4WD control)
Motor 1 (Left Front) IN1	GPIO 18	Forward control
Motor 1 (Left Front) IN2	GPIO 19	Backward control
Motor 2 (Right Front) IN1	GPIO 21	Forward control
Motor 2 (Right Front) IN2	GPIO 22	Backward control
Speed Control 1 (PWM)	GPIO 5	PWM for Motor 1
Speed Control 2 (PWM)	GPIO 4	PWM for Motor 2
DC Drill Motor Control		(Using Relay or MOSFET)
Relay / MOSFET Signal	GPIO 23	Turns drill ON/OFF
Bluetooth Serial	Built-in	Used for remote control
Power Connections		(External Power Source)
L298N VCC	Battery (7.4V-12V)	Power for motors
L298N GND	GND	Common ground with ESP32
ESP32 VIN	Battery 5V Regulator	Powers ESP32

PINOUT DIAGRAM

ESP32 + Servo Motor



PINOUT DIAGRAM

ESP32 + Servo Motor

ESP32 PIN	Servo Motor
3.3v	VCC
GND	GND
D4	Signal

CONCLUSION

In this project, we successfully got the 4-wheel chassis to function and managed to make individual sensors work. However, we faced several challenges that prevented full integration. One major issue was an unstable power supply due to the absence of a stationary source, leading to possible overloads. Using a single 12V battery pack to power multiple 12V outputs may have contributed to this problem. Additionally, our ESP32 module burned out twice, likely due to power inconsistencies or mismanagement. Another key factor was our limited experience in designing large and complex circuits within a compact space, which made proper wiring and power distribution difficult.

Despite these setbacks, the project provided valuable learning experiences in hardware troubleshooting, power management, and circuit design, which will help in future developments.

THANK YOU FOR LISTENING

